Politécnico Castelo Branco
Escola Superior de Tecnologia

Licenciatura em Engenharia Informática

Algoritmos e Estrutura de Dados

Trabalho Prático 1

1º Ano – 2º Semestre
Ano letivo 2025/2026

Número	Nome
20231405	Joel da Fonseca Malacas
20231269	Gonçalo Santos Cabrito

Autoavaliação

Assinalar com um **X** a situação de cada critério de avaliação/especificação da aplicação:

Funcionalidade	☹	😐	😊
Ler o ficheiro e carregar para o vetor			X
Implementação dos Algoritmos de Ordenação			X
Testar os Algoritmos de Ordenação			X
Testar o Algoritmo de Pesquisa Linear			X
Testar o Algoritmo de Pesquisa Binária			X
Discussão dos resultados			X

Implementação

```java
public static String[] LerFicheiro(String caminho) {

        File ficheiro=new File(caminho);

        Scanner leitor=null;

        try {
            leitor = new Scanner(ficheiro);
        } catch (FileNotFoundException e) {
            System.out.println("Ficheiro não encontrado");
            System.out.println("Erro: " + e); //DEBUG
        }

        //While ⟹ Contar as palavras
        int count = 0;
        while(leitor.hasNext()){
            leitor.next();
            count++;
        }

        leitor.close();

        //═══========================

        String[] palavras = new String[count];

        try {
            leitor = new Scanner(ficheiro);
        } catch (FileNotFoundException e) {
            System.out.println("Erro: " + e);
            return new String[0];
        }

        //While ⟹ Colocar as palavras no vetor Aux
        int i = 0;
        while (leitor.hasNext()) {
            palavras[i] = leitor.next();
            i++;
        }

        leitor.close();

        return palavras;
    }
```

Método “LerFicheiro”, da Classe Ficheiro.java que no primeiro “Try”, tenta ler o ficheiro que será inserido no parâmetro do método e após com o “While” irá ser contado as palavras para que de seguida seja inicializado um array de String com o tamanho definido no contador de palavras. Após a inicialização do Array “palavras”, irá percorrer a cada palavra do ficheiro e irá armazenar no Array.

```java
private long inicio, fim;
    private String[] auxOrdenado;

    public long ordenaPermutacao(String[] oVector) {
        inicio = System.currentTimeMillis();
        auxOrdenado = new String[oVector.length];


        boolean ordenado = false;
        int i = 0;
        while (i < oVector.length - 1 && !ordenado) {
            ordenado = true;
            for (int j = oVector.length - 1; j > i; j--) {
                if (oVector[j - 1].compareTo(oVector[j]) > 0) {
                    String aux = oVector[j - 1];
                    oVector[j - 1] = oVector[j];
                    oVector[j] = aux;
                    ordenado = false;
                }
            }
            i++;
        }

        fim=System.currentTimeMillis();

        for (int l = 0; l < oVector.length; l++) {
            auxOrdenado[l] = oVector[l];
        }
        return fim - inicio;
    }

    public String[] getAuxOrdenado () {
        return auxOrdenado;
    }
```

codesnap.dev

Na Classe Permutacao.java além do código fornecido do algoritmo da permutação, após a ordenação é efetuado um ciclo for para armazenar num novo vetor auxiliar os elementos ordenados, para após isso criar um método para retornar esse mesmo vetor auxiliar com os vetores ordenados.
Para que na classe principal ser chamado o método para um vetor auxiliar armazenar esse mesmo retorno. Para a 3 bateria de testes.

```java
public class PesquisaLinear {

    private long inicio,fim;
    private long[] ResultsExis, ResultsIns;

    private void ProcuraLinear(String[] v, String palavra) {
        for(int i = 0; i < v.length; i++) {
            if (Objects.equals(v[i], palavra)) {
                return;
            }
        }
    }

    public void Existente(String[] palavras, String[] Sucesso) {
        ResultsExis = new long[Sucesso.length];

        for (int i = 0; i < Sucesso.length; i++) {
            inicio = System.currentTimeMillis();
            ProcuraLinear(palavras, Sucesso[i]);
            fim = System.currentTimeMillis();

            ResultsExis[i] = fim - inicio;
        }
    }

    public void Inexistente(String[] palavras, String[] Insucesso) {
        ResultsIns = new long[Insucesso.length];

        for (int j = 0; j < Insucesso.length; j++) { //Palavras inexistentes na wordlist
            inicio = System.currentTimeMillis();
            ProcuraLinear(palavras, Insucesso[j]);
            fim = System.currentTimeMillis();

            ResultsIns[j] = fim - inicio;
        }
    }

    public String PrintSucesso() {
        String result = "";
        for (int i = 0; i < ResultsExis.length; i++) {
            result += "v[" + i + "] = " + ResultsExis[i] + "ms\n";
        }
        return result;
    }

    public String PrintInsucesso() {
        String result = "";
        for (int j = 0; j < ResultsIns.length; j++) {
            result += "v[" + j + "] = " + ResultsIns[j] + "ms\n";
        }
        return result;
    }
}
```

codesnap.dev

Na classe pesquisa linear, é inicializado dois arrays um para resultados existentes e um para inexistentes, que é armazenado os respetivos tempos seja de sucesso ou incusesso na sua procura. E também existe dois métodos para apenas imprimir os seus elementos do array.
O mesmo método e lógica usada na pesquisa binária.

```java
public static void main(String[] args) {
        //Constante do caminho do ficheiro .txt
        final String caminhoFicheiro = "palavras.txt";

        //Instâncias
        Selecao selecao = new Selecao();
        Insercao insercao = new Insercao();
        Permutacao permutacao = new Permutacao();
        MergeSort mergeSort = new MergeSort();
        QuickSort quickSort = new QuickSort();
        PesquisaLinear pesquisaLinear = new PesquisaLinear();
        PesquisaBinaria pesquisaBinaria = new PesquisaBinaria();

        String[] palavras = Ficheiro.LerFicheiro(caminhoFicheiro);

        //========Primeira Bateria de Testes==========
        long resultMerge = mergeSort.MergeSort(palavras);
        long resultQuick = quickSort.ordenaQuickSort(palavras);
        long resultSelecao = selecao.ordenaSelecao(palavras);
        long resultInsercao = insercao.ordenaInsercao(palavras);
        long resultPermutacao = permutacao.ordenaPermutacao(palavras);

        //========Segunda Bateria de Testes==========
        String[] Sucesso =  {"arruinai", "capitel", "curso", "eslavo", "gravataria", "longo", "obtenhais", "progenitor", "seria", "vaca"};
        String[] Insucesso = {"algoritmo", "condicional", "direita", "esquerda", "grande", "livros", "prata", "silencio", "verde", "xarões"};

        pesquisaLinear.Existente(palavras, Sucesso);
        pesquisaLinear.Inexistente(palavras, Insucesso);

        String linearSucesso2 = pesquisaLinear.PrintSucesso();
        String linearInsucesso2 = pesquisaLinear.PrintInsucesso();

        //========Terceira Bateria de Testes==========
        String[] palavrasOrdenado = permutacao.getAuxOrdenado();

        pesquisaLinear.Existente(palavrasOrdenado,Sucesso);
        pesquisaLinear.Inexistente(palavrasOrdenado,Insucesso);

        String linearSucesso3 = pesquisaLinear.PrintSucesso();
        String linearInsucesso3 = pesquisaLinear.PrintInsucesso();

        pesquisaBinaria.Existente(palavrasOrdenado, Sucesso);
        pesquisaBinaria.Inexistente(palavrasOrdenado, Insucesso);

        String binariaSucesso3 = pesquisaBinaria.PrintSucesso();
        String binariaInsucesso3 = pesquisaBinaria.PrintInsucesso();
```

codesnap.dev

Na classe principal Main.java, é inicializado uma constante com o nome do ficheiro .txt desejado. Após isso é instanciado as respetivas classes dos algoritmos de ordenação, e é armazenado no array palavras o retorno do método LerFicheiro, que armazena a sua wordlist no array.

De seguida é efetuado a primeira bateria de testes que apenas armazena o resultado de cada algoritmo de ordenação.

Na segunda bateria de testes, é inicializado dois arrays com palavras existentes e inexistentes na wordlist, para que seja efetuado as respetivas pesquisas lineares e imprimir o resultado das mesmas.

Na terceira bateria de testes, é inicializado e armazenado o retorno das palavras ordenadas com o método anunciado e explicado na segunda imagem para ser usar nos métodos das classes de pesquisa linear e binária.

Resultados Obtidos

Algoritmos de Ordenação

Tempo de Execução (ms)				
Seleção	Inserção	Permutação	Merge Sort	Quick Sort
258	1	2	18	12

Vetor desordenado

	Tempo de Execução (ms) - Teste Positivo									
	arruinai	capitel	curso	eslavo	gravataria	longo	obtenhais	progenitor	seria	vaca
Pesquisa Linear	0	1	0	0	1	0	1	0	0	0

	Tempo de Execução (ms) - Teste Negativo									
	algoritmo	condicional	direita	esquerda	grande	livros	prata	silencio	verde	xarões
Pesquisa Linear	2	0	0	1	0	0	0	0	0	1

Vetor ordenado

	Tempo de Execução (ms) - Teste Positivo									
	arruinai	capitel	curso	eslavo	gravataria	longo	obtenhais	progenitor	seria	vaca
Pesquisa Linear	0	0	1	0	0	0	0	1	0	0
Pesquisa Binária	0	0	0	0	0	0	0	0	0	0

	Tempo de Execução (ms) - Teste Negativo									
	algoritmo	condicional	direita	esquerda	grande	livros	prata	silencio	verde	xarões
Pesquisa Linear	0	0	0	1	0	0	0	0	0	0
Pesquisa Binária	0	0	0	0	0	0	0	0	0	0

Análise

Os testes realizados permitiram comparar o desempenho de cada um dos algoritmos de pesquisa e ordenação em dois cenários diferentes, nomeadamente com vetores desordenados e ordenados.

Com os resultados obtidos nos algoritmos de ordenação é possível ver que:

- O algoritmo de Seleção apresentou o pior desempenho, devido à sua complexidade este demonstra ser pouco eficiente quando se trata de um conjunto de dados maior;
- A Inserção apresentou o melhor desempenho, esta demonstra ser mais simples e eficiente do que a Seleção;
- A Permutação também teve um ótimo desempenho neste teste apresentando um valor apenas 1ms mais alto que a Inserção, embora noutros testes realizados é possível reparar que a Permutação e a Inserção mostram velocidades bastante parecidas ou iguais dependendo da maneira como o vetor esteja composto;
- O MergeSort teve um bom desempenho, porém não é tão eficiente quanto os dois algoritmos anteriores;
- O QuickSort apresentou um bom desempenho, mas apesar de ser identificado como o algoritmo mais rápido de ordenação, este mostra que não é o melhor nesta experiência.

Quanto aos algoritmos de pesquisa foi possível verificar que a Pesquisa Linear apresentou tempos quase nulos nos dois tipos de vetor, sendo possível verificar que existem tempos mais elevados no insucesso visto que o algoritmo tem de percorrer o vetor todo.

Já na Pesquisa Binária, que é aplicada apenas no caso do vetor ordenado pois é o único caso em que pode ser utilizada, apresentou apenas valores de 0 milissegundos, mostrando ser consistente e mais rápida que a Pesquisa Linear tantos nos casos de sucesso como de insucesso.

Durante este teste foi necessário recorrer ao uso de nanossegundos devido aos algoritmos de pesquisa apresentarem valores extremamente baixos impedindo que fosse possível uma verificação mais exata da performance destes algoritmos. Através nos nanossegundos, foi possível confirmar que a Pesquisa Linear teve um crescimento progressivo de tempo conforme a posição do elemento aumenta, que a Pesquisa Binária apresentou tempos em nanossegundos muito próximos de 0 e permitiu verificar que o programa estava a funcionar da maneira que era pretendida.

Concluindo, para este tipo de teste a Inserção e a Permutação foram os algoritmos de ordenação que se destacaram sendo ainda mais eficientes do que o QuickSort enquanto nos algoritmos de pesquisa a Pesquisa Binária foi a mais rápida no vetor ordenado demonstrando ser a melhor opção

Código Fonte

```
import Algoritmos.*;

/*
   * Classe Principal Main.java
   * Onde vai ser executado as 3 Baterias de testes com as respetivas classes
   * e mostradas as respetivas saídas
*/

public class Main {
   public static void main(String[] args) {
      //Constante do caminho do ficheiro .txt
      final String caminhoFicheiro = "palavras.txt";

      //Instâncias
      Selecao selecao = new Selecao();
      Insercao insercao = new Insercao();
      Permutacao permutacao = new Permutacao();
      MergeSort mergeSort = new MergeSort();
      QuickSort quickSort = new QuickSort();
      PesquisaLinear pesquisaLinear = new PesquisaLinear();
      PesquisaBinaria pesquisaBinaria = new PesquisaBinaria();

      String[] palavras = Ficheiro.LerFicheiro(caminhoFicheiro);

      //==========Primeira Bateria de Testes==========
      long resultMerge = mergeSort.MergeSort(palavras);
      long resultQuick = quickSort.ordenaQuickSort(palavras);
      long resultSelecao = selecao.ordenaSelecao(palavras);
      long resultInsercao = insercao.ordenaInsercao(palavras);
      long resultPermutacao = permutacao.ordenaPermutacao(palavras);

      //==========Segunda Bateria de Testes==========
      String[] Sucesso =  {"arruinai", "capitel", "curso", "eslavo", "gravataria",
"longo", "obtenhais", "progenitor", "seria", "vaca"};
      String[] Insucesso = {"algoritmo", "condicional", "direita", "esquerda",
"grande", "livros", "prata", "silencio", "verde", "xarões"};

      pesquisaLinear.Existente(palavras, Sucesso);
      pesquisaLinear.Inexistente(palavras, Insucesso);

      String linearSucesso2 = pesquisaLinear.PrintSucesso();
      String linearInsucesso2 = pesquisaLinear.PrintInsucesso();

      //==========Terceira Bateria de Testes==========
      String[] palavrasOrdenado = permutacao.getAuxOrdenado();

      pesquisaLinear.Existente(palavrasOrdenado,Sucesso);
```

```
        pesquisaLinear.Inexistente(palavrasOrdenado,Insucesso);

        String linearSucesso3 = pesquisaLinear.PrintSucesso();
        String linearInsucesso3 = pesquisaLinear.PrintInsucesso();

        pesquisaBinaria.Existente(palavrasOrdenado, Sucesso);
        pesquisaBinaria.Inexistente(palavrasOrdenado, Insucesso);

        String binariaSucesso3 = pesquisaBinaria.PrintSucesso();
        String binariaInsucesso3 = pesquisaBinaria.PrintInsucesso();

        //==========Output's==========
        System.out.println("Tempo de ordenação Seleção: " + resultSelecao + "ms");
        System.out.println("Tempo de ordenação Inserção: " + resultInsercao +
"ms");
        System.out.println("Tempo de ordenação Permutação: " + resultPermutacao
+ "ms");
        System.out.println("Tempo de ordenação MergeSort: " + resultMerge +
"ms");
        System.out.println("Tempo de ordenação QuickSort:" + resultQuick + "ms
\n\n");

        System.out.println("==========Pesquisa Linear==========");
        System.out.println("==========Sucesso==========\n" + linearSucesso2);
        System.out.println("==========Insucesso==========\n" +
linearInsucesso2);

        System.out.println("==========Vetor Ordenado==========");
        System.out.println("==========Pesquisa Linear==========");
        System.out.println("=============Sucesso==========\n" +
linearSucesso3);
        System.out.println("==========Insucesso==========\n" +
linearInsucesso3);
        System.out.println("\n==========Pesquisa Binária==========");
        System.out.println("=============Sucesso==========\n" +
binariaSucesso3);
        System.out.println("==========Insucesso==========\n" +
binariaInsucesso3);
    }
}
```

```
package Algoritmos;

import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class Ficheiro {

   /*
      * Método que lê wordlist (.txt) e devolve um array de strings
   */

   public static String[] LerFicheiro(String caminho) {

      File ficheiro=new File(caminho);

      Scanner leitor=null;

      try {
         leitor = new Scanner(ficheiro);
      } catch (FileNotFoundException e) {
         System.out.println("Ficheiro não encontrado");
         System.out.println("Erro: " + e); //DEBUG
      }

      //While ==> Contar as palavras
      int count = 0;
      while(leitor.hasNext()){
         leitor.next();
         count++;
      }

      leitor.close();

      //============================

      String[] palavras = new String[count];

      try {
         leitor = new Scanner(ficheiro);
      } catch (FileNotFoundException e) {
         System.out.println("Erro: " + e);
         return new String[0];
      }

      //While ==> Colocar as palavras no vetor Aux
      int i = 0;
      while (leitor.hasNext()) {
         palavras[i] = leitor.next();
```

```
            i++;
        }

        leitor.close();

        return palavras;
    }

}
```

```
package Algoritmos;

/*
   * Algoritmo Ordenação por Inserção,
   * vai colocando cada elemento na sua posição correta
   * considerando-se apenas os que estão à esquerda do elemento atual
   * Ao inserir um elemento os maiores são deslocados para a direita
*/

public class Insercao {

   private long inicio, fim;

   public long ordenaInsercao(String[] oVector) {
      inicio = System.currentTimeMillis();

      for (int i = 1; i < oVector.length; i++) {
         String x = oVector[i];
         int j = i;
         while (j > 0 && oVector[j - 1].compareTo(x) > 0) {
            oVector[j] = oVector[j - 1];
            j--;
         }
         oVector[j] = x;
      }

      fim = System.currentTimeMillis();
      return fim - inicio;
   }
}
```

```
package Algoritmos;

/*
   * Classe MergeSort.java
   * Algoritmo de Ordenação por MergeSort
   * é um algoritmo recursivo, divide
   * o vetor a meio, formando dois vetores:
      * Ordena-os separadamente,
      * Reagrupa-os num novo vetor
   * Para ordenar os vetores resultantes aplica o mesmo raciocínio
   * Obriga a crirar um novo vetor auxiliar devido ao reagrupamento
*/

public class MergeSort {

   private long inicio,fim;

   public long MergeSort(String[] v) {
      inicio = System.currentTimeMillis();

      String[] aux = new String[v.length];
      MergeSortRec(v, aux, 0, v.length - 1);

      fim=System.currentTimeMillis();

      return fim-inicio;
   }

   private void MergeSortRec(String[] v, String[] vAux, int esq, int dir) {
      if (esq >= dir) {
         return;
      }

      int centro = (esq + dir) / 2;
      MergeSortRec(v, vAux, esq, centro);
      MergeSortRec(v, vAux, centro + 1, dir);

      Reagrupar(v, vAux, esq, centro + 1, dir);
   }

   private void Reagrupar(String[] v, String[] vAux, int i1, int i2, int fim2) {
      int fim1 = i2 - 1;
      int inicio = i1;
      int iAux = i1;

      while (i1 <= fim1 && i2 <= fim2) {
         if (v[i1].compareTo(v[i2]) < 0) {
            vAux[iAux] = v[i1];
            i1++;
```

```
            } else {
                vAux[iAux] = v[i2];
                i2++;
            }
            iAux++;
        }

        for (; i1 <= fim1; i1++, iAux++)
            vAux[iAux] = v[i1];
        for (; i2 <= fim2; i2++, iAux++)
            vAux[iAux] = v[i2];
        for (int i = inicio; i <= fim2; i++)
            v[i] = vAux[i];
    }

    public void PrintVetor(String[] v) {
        for (int i = 0; i < v.length; i++) {
            System.out.println("v[" + i + "]=" + v[i]);
        }
    }
}
```

```
package Algoritmos;

/*
   * Classe Permutacao.java
   * Algoritmo de Ordenação por Permutação
   * Neste caso percorre-se o vetor, do último para o primeiro:
      *Comparando elementos consecutivos dois a dois
      * Trocando-os se estiverem na ordem inversa
   *Após a primeira iteração o menor valor estará na primeira posição
      * Pelo que as repetições seguintes, deverão ter em conta apenas os restantes
      * elementos e assim sucessivamente
*/

public class Permutacao {

   private long inicio, fim;
   private String[] auxOrdenado;

   public long ordenaPermutacao(String[] oVector) {
      inicio = System.currentTimeMillis();
      auxOrdenado = new String[oVector.length];

      boolean ordenado = false;
      int i = 0;
      while (i < oVector.length - 1 && !ordenado) {
         ordenado = true;
         for (int j = oVector.length - 1; j > i; j--) {
            if (oVector[j - 1].compareTo(oVector[j]) > 0) {
               String aux = oVector[j - 1];
               oVector[j - 1] = oVector[j];
               oVector[j] = aux;
               ordenado = false;
            }
         }
         i++;
      }

      fim=System.currentTimeMillis();

      for (int l = 0; l < oVector.length; l++) {
         auxOrdenado[l] = oVector[l];
      }
      return fim - inicio;
   }

   public String[] getAuxOrdenado () {
      return auxOrdenado;
   }
}
```

```
package Algoritmos;

import java.util.Objects;

/*
   * Classe PesquisaBinaria.java
   * Algoritmo de Pesquisa Binária
   * Na pesquisa binária o vetor é divido em 2 zonas
   * A divisão pode ser feita de qualquer maneira, mas
   * os melhores resultados (em média) obtêm-se quando
   * se divide o vetor (e as zonas seguintes) em duas partes iguais
   * Comparando elemento a procurar (x) com o elemento divisor
   * escolhe-se a próxima zona a dividir:
      *Se x for menor procurar na zona da esquerda
      * Se x for maior procurar na zona da direita
   * E assim sucessivamente até encontrar o valor ou a zona estar vazia
*/

public class PesquisaBinaria {

   private long inicio,fim;
   private long[] ResultsExis, ResultsIns;

   private void ProcuraBinaria (String[] v, String palavra) {
      boolean encontrou = false;
      int esq = 0, dir = v.length - 1, meio = 0;

      while (esq <= dir && !encontrou) {
         meio = (esq + dir) / 2;

         if (Objects.equals(v[meio], palavra))
            encontrou = true;
         else if (v[meio].compareTo(palavra) < 0)
            esq = meio + 1;
         else
            dir = meio - 1;
      }
   }

   public void Existente(String[] palavras, String[] Sucesso) {
      ResultsExis = new long[Sucesso.length];

      for (int i = 0; i < Sucesso.length; i++) {
         inicio = System.currentTimeMillis();
         ProcuraBinaria(palavras, Sucesso[i]);
         fim = System.currentTimeMillis();

         ResultsExis[i] = fim - inicio;
      }
```

```
    }

    public void Inexistente(String[] palavras, String[] Insucesso) {
        ResultsIns = new long[Insucesso.length];

        for (int j = 0; j < Insucesso.length; j++) { //Palavras inexistentes na wordlist
            inicio = System.currentTimeMillis();
            ProcuraBinaria(palavras, Insucesso[j]);
            fim = System.currentTimeMillis();

            ResultsIns[j] = fim - inicio;
        }
    }

    public String PrintSucesso() {
        String result = "";
        for (int i = 0; i < ResultsExis.length; i++) {
            result += "v[" + i + "] = " + ResultsExis[i] + "ms\n";
        }
        return result;
    }

    public String PrintInsucesso() {
        String result = "";
        for (int j = 0; j < ResultsIns.length; j++) {
            result += "v[" + j + "] = " + ResultsIns[j] + "ms\n";
        }
        return result;
    }
}
```

```
package Algoritmos;

/*
   * Classe PesquisaLinear
   * Algoritmo de Pesquisa Linear
   * Nesta procura o vetor é percorrido, elemento a elemento,
   * desde o primeiro até ao último
   * Termina-se a procura quando ocorre uma de duas situações:
      *O elemento é encontrado
      *todo o vetor foi pesquisado e não se encontrou o elemento
*/

import java.util.Objects;

public class PesquisaLinear {

   private long inicio,fim;
   private long[] ResultsExis, ResultsIns;

   private void ProcuraLinear(String[] v, String palavra) {
      for(int i = 0; i < v.length; i++) {
         if (Objects.equals(v[i], palavra)) {
            return;
         }
      }
   }

   public void Existente(String[] palavras, String[] Sucesso) {
      ResultsExis = new long[Sucesso.length];

      for (int i = 0; i < Sucesso.length; i++) {
         inicio = System.currentTimeMillis();
         ProcuraLinear(palavras, Sucesso[i]);
         fim = System.currentTimeMillis();

         ResultsExis[i] = fim - inicio;
      }
   }

   public void Inexistente(String[] palavras, String[] Insucesso) {
      ResultsIns = new long[Insucesso.length];

      for (int j = 0; j < Insucesso.length; j++) { //Palavras inexistentes na wordlist
         inicio = System.currentTimeMillis();
         ProcuraLinear(palavras, Insucesso[j]);
         fim = System.currentTimeMillis();

         ResultsIns[j] = fim - inicio;
      }
```

```
    }

    public String PrintSucesso() {
        String result = "";
        for (int i = 0; i < ResultsExis.length; i++) {
            result += "v[" + i + "] = " + ResultsExis[i] + "ms\n";
        }
        return result;
    }

    public String PrintInsucesso() {
        String result = "";
        for (int j = 0; j < ResultsIns.length; j++) {
            result += "v[" + j + "] = " + ResultsIns[j] + "ms\n";
        }
        return result;
    }
}
```

```
package Algoritmos;

/*
   * Classe QuickSort
   * Algoritmo de Ordenação QuickSort
   * é um algoritmo recursivo
   * é o algoritmo mais rápido de ordenação
   * escolhe primeiramente um valot, também chamado de pivot
   * divide o vetor em duas partes:
      * a primeira parte contém elementos menores que o pivot
      * a segunda parte contém elementos maiores que o pivot
      * o pivot é colocado no meio das duas partes
   * para ordenar as partes daí resultantes aplica-se o mesmo raciocínio
*/

public class QuickSort {

   private long inicio,fim;

   public long ordenaQuickSort(String[] v) {
      inicio = System.currentTimeMillis();

      quickSortRec(v, 0, v.length - 1);

      fim=System.currentTimeMillis();

      return fim-inicio;
   }

   private void quickSortRec(String[] v, int ini, int fim) {
      if (ini >= fim) {
         return;
      }

      String pivot = pivotMediana(v, ini, fim);
      int i = DivideMaiores(v, ini, fim, pivot);

      quickSortRec(v, ini, i - 1);
      quickSortRec(v, i + 1, fim);
   }

   private int DivideMaiores(String[] v, int ini, int fim, String pivot) {
      int i = ini;
      int j = fim - 1;

      while (true) {
         while (v[++i].compareTo(pivot) < 0);
         while (j > 0 && v[--j].compareTo(pivot) > 0);
```

```
            if (i < j) {
                troca(v, i, j);
            } else {
                break;
            }
        }

        troca(v, i, fim - 1);
        return i;
    }

    private String pivotMediana(String[] v, int ini, int fim) {
        int centro = (ini + fim) / 2;
        if (v[ini].compareTo(v[centro]) > 0)
            troca(v, ini, centro);
        if (v[ini].compareTo(v[fim]) > 0)
            troca(v, ini, fim);
        if (v[centro].compareTo(v[fim]) > 0)
            troca(v, centro, fim);

        troca(v, centro, fim - 1);
        return v[fim - 1];
    }

    private void troca(String[] v, int i, int j) {
        String aux = v[i];
        v[i] = v[j];
        v[j] = aux;
    }

    public void PrintVetor(String[] v) {
        for (int i = 0; i < v.length; i++) {
            System.out.println("v[" + i + "] = " + v[i]);
        }
    }

}
```

```
package Algoritmos;
 /*
   *Algoritmo Ordenação por Seleção
   * procura o menor elemento do vetor
   * troca o menor elemento com o primeiro elemento
   * repete o processo até o vetor estar ordenado
  */
public class Selecao {
   private long inicio, fim;

   public long ordenaSelecao(String[] oVector) {
      inicio = System.currentTimeMillis();

      for (int i = 0; i < oVector.length; i++) {
         String min = oVector[i];
         int minIndex = i;
         for (int j = i + 1; j < oVector.length; j++)
            if (oVector[j].compareTo(min) < 0) {
               min = oVector[j];
               minIndex = j;
            }

         String aux = oVector[minIndex];
         oVector[minIndex] = oVector[i];
         oVector[i] = aux;
      }

      fim=System.currentTimeMillis();

      return fim - inicio;
   }
}
```